Pseudo-Biquality Learning with Confident Learning: An Empirical Study of Label-Noise Handling without Human Verification

codemajin

October 2026

Abstract

Biquality learning handles label noise by using both a small trusted dataset whose labels are guaranteed to be correct and a large untrusted dataset that may contain noise. Preparing the trusted data, however, usually requires human verification. In this study we automatically build a trusted/untrusted split from the label-error flags and quality scores produced by cleanlab, an implementation of Confident Learning (pseudo-biquality learning), and feed it to the algorithms of biquality-learn. We compare this approach with training on the data as is, pruning with cleanlab (including cleanlab's standard CleanLearning), and using the quality scores as sample weights or features. We evaluate on 90 conditions (3 tabular datasets, 3 kinds of synthetic noise, 2 noise rates, and 5 seeds) with two base models, logistic regression and gradient boosting. Pseudo trusted data combined with IRBL achieved the highest mean accuracy among the methods that use no human input (+2.73 pt over the baseline for LR and +3.56 pt for HGB). Its advantage over simple pruning, however, was only 0.3–0.5 pt, at about three times the training time. CleanLearning, which adds per-class reweighting, did not outperform simple pruning. In contrast, the transition-matrix methods GLC, Plugin, and IRLNL lost up to 37 pt of accuracy when combined with pseudo trusted data. We confirm that a split based on cleanlab's flags concentrates mislabeled samples in the untrusted set, which makes the diagonal of the transition matrix severely underestimated. Choosing the samples with the highest quality scores as trusted lowered accuracy even though it lowered the error rate of the trusted set. As a control, when we simulate human verification of 30% of the training data, IRBL outperforms the best automatic method by +3.91 pt (LR) and +5.00 pt (HGB) on average.

1 Introduction

Labels in supervised learning often contain errors caused by annotation mistakes or ambiguity (label noise) [1, 2]. Approaches to label noise fall into two broad families. One detects mislabeled samples and removes or corrects them; a representative example is Confident Learning [3] and its implementation cleanlab. The other assumes noise and corrects the training procedure; among these, biquality learning [4] uses a small trusted dataset whose labels are guaranteed to be correct as a guide for reweighting or correcting the loss of a large untrusted dataset. biquality-learn [5] is a library that implements the algorithms of this framework with a scikit-learn [6] compatible interface.

The weakness of biquality learning is that building the trusted data requires human verification. A natural idea is therefore to skip the human step by treating the samples that cleanlab judges to be "error-free" as trusted. cleanlab's judgment, however, relies on model predictions, so the resulting trusted set is biased toward "samples the model can predict confidently". Moreover, many biquality methods implicitly assume that the trusted and untrusted data are drawn from the same distribution. Whether this combination actually works is therefore not obvious.

The goal of this study is to answer the following questions under the condition that no human input is used at all.

RQ1
Does feeding pseudo trusted/untrusted sets built from cleanlab's output into biquality-learn improve accuracy over pruning with cleanlab?

RQ2
How does the way the pseudo trusted set is chosen (flags only, top quality scores, or random) affect the results?

RQ3
Is it effective to use cleanlab's quality scores as sample weights or features?

RQ4
What mechanisms produce these results?

Our contributions are as follows.

- We combine three pseudo-biquality splitting schemes with six biquality-learn algorithms (plus training on the trusted set only) and compare them with pruning, CleanLearning, weighting, and feature augmentation on 90 conditions $\times$ 2 models.
- We show that weighting methods (IRBL) match or exceed pruning while transition-matrix methods break down badly, and we explain why from measurements of the estimated transition matrices, the error rates of the untrusted sets, and the IRBL weights.
- With control experiments that provide genuinely clean trusted sets of two sizes (10% and 30%), we separate the degradation caused by pseudo trusted data, the effect of the trusted-set size, and the weaknesses of the methods themselves.

2 Related Work

Learning with label noise Classification under label noise has long been studied. Noise is broadly divided into class-conditional noise, whose flip probabilities depend only on the class, and instance-dependent noise, which also depends on the input [1, 2].

Detecting mislabeled examples Confident Learning [3] estimates the joint distribution of noisy and true labels (the confident joint) from cross-validated out-of-sample predicted probabilities and the given labels, and enumerates candidate label errors. cleanlab additionally provides label quality scores such as the predicted probability of the given label (self-confidence), and `CleanLearning`, which removes the candidate errors and retrains with weights based on the per-class noise rates. George et al. [7] organize many mislabeled-example detectors, including Confident Learning, under a unified view as "probes" of trained models and benchmark them on both completely-at-random and not-at-random noise. We evaluate the outputs of such detectors not by their detection accuracy but by their usefulness as input to downstream biquality learning.

Selection bias of confidence-based filtering Swayamdipta et al. [8] categorize samples as "easy-to-learn", "ambiguous", or "hard-to-learn" from the model's confidence during training and its variability, and show that mislabeled examples concentrate in the hard-to-learn region while ambiguous examples contribute to generalization. Their observation that selecting only high-confidence samples biases the selection toward easy examples bears directly on how we choose the trusted set (RQ2).

Biquality learning In biquality learning [4], the training data consist of a trusted set $\mathcal{T}$ and an untrusted set $\mathcal{U}$. The methods implemented in biquality-learn fall into two families. Weighting methods assign a weight to each sample of $\mathcal{U}$ to bring it closer to the distribution of $\mathcal{T}$: IRBL [9], K-PDR, which estimates density ratios with a probabilistic classifier [4, 10], and K-KMM, which uses kernel mean matching [4, 11]. Transition-matrix methods estimate the label transition matrix $T_{ij} = P(\tilde{y} = j \mid y = i)$ and correct with it: GLC corrects the loss [12], Plugin corrects only at prediction time [13], and IRLNL derives importance weights from the transition matrix [14].

Of the three transition-matrix methods, the original Plugin and IRLNL estimate T from noisy data alone, for example via anchor points. biquality-learn adapts them to the biquality setting by estimating T with the same `gold` estimator as GLC. That is, a probabilistic classifier trained on $\mathcal{U}$ is evaluated on $\mathcal{T}$, and for each label i in $\mathcal{T}$ the predicted probabilities of the noisy labels are averaged to form row i of T. IRLNL uses the ratio of the clean class posterior obtained from this T to the noisy class posterior as the weights of $\mathcal{U}$, and gives weight 1 to $\mathcal{T}$. All three methods therefore assume that $\mathcal{U}$ is representative of the noise distribution of the whole dataset, and they all break down together when this assumption fails.

3 Method

3.1 Problem setting

The labels $\tilde{y}_i \in \{1, \ldots, K\}$ of the training data $\mathcal{D} = \{(x_i, \tilde{y}_i)\}_{i=1}^n$ contain errors, and the true labels y_i are not observed. Without any human verification, we learn a classifier from $\mathcal{D}$ alone.

3.2 Output of cleanlab

We train the base model with stratified 5-fold cross-validation to obtain out-of-sample predicted probabilities $\hat{p}_i \in \Delta^{K-1}$. Passing these to cleanlab yields the set of samples flagged as label errors, $\mathcal{I} \subset \{1, \ldots, n\}$ (`find_label_issues` with default settings), and label quality scores s_i. We use two scores: self-confidence $s_i = \hat{p}_{i,\tilde{y}_i} \in [0, 1]$ and normalized margin $s_i = (\hat{p}_{i,\tilde{y}_i} - \max_{k \neq \tilde{y}_i} \hat{p}_{i,k} + 1)/2 \in [0, 1]$. The latter follows cleanlab's definition and maps the margin to $[0, 1]$. Wherever s_i is used outside `cl_weight/*` (`topconf` and `cl_feature`), we use self-confidence.

3.3 Methods compared

All methods learn from $\mathcal{D}$ alone. The numbers denote groups of methods and correspond to the groups in Table 1.

(1) `baseline`: train on $\mathcal{D}$ as is.

(2) `cl_prune`: train on $\mathcal{D} \setminus \mathcal{I}$ with uniform weights.
`cl_cleanlearning`: use cleanlab's `CleanLearning`. Because it receives the same $\hat{p}_i$, it removes exactly the same samples as `cl_prune`, and then trains on the remaining samples with class k weighted by the inverse of the diagonal of the noise matrix estimated from the confident joint, $1/\max(\hat{P}(\tilde{y} = k \mid y = k), 10^{-3})$. The only difference between the two is this per-class reweighting.

(3) `bq/<split>/<algo>`: build a pseudo trusted/untrusted split (Section 3.4) and train with a biquality-learn algorithm. The algorithms are IRBL, K-PDR, K-KMM, GLC, Plugin, IRLNL, and `trusted_only`, which trains on $\mathcal{T}$ alone with uniform weights. All transition-matrix methods use the `gold` estimator. With the `naive` split, $\mathcal{T} = \mathcal{D} \setminus \mathcal{I}$, so `bq/naive/trusted_only` is identical to `cl_prune` and is omitted from the tables.

(4) `cl_weight/*`: train on $\mathcal{D}$ with s_i as sample weights.
`cl_feature`: train on the features $[x_i, s_i]$ and feed $s = 1$ at inference. x_i is standardized, while s_i (self-confidence) is appended unstandardized in $[0, 1]$. Because s is not available at inference, we condition on the upper end of its range (the most reliable label) instead.

As references we report `oracle_clean`, trained on the true labels, and `oracle_trusted10/irbl` and `oracle_trusted30/irbl`, which restore the true labels of 10% or 30% of the training samples chosen uniformly at random (without stratification), treat them as trusted and all the rest as untrusted, and train with IRBL. The latter correspond to the intended use of biquality-learn: "verify a small amount by hand". We chose 10% as a realistic verification budget and 30% to match the trusted-set size of the pseudo splits ($\tau = 0.3$). Both are built from the same uniform random draws, so the 10% trusted set is contained in the 30% one.

3.4 Pseudo trusted/untrusted splits

Let $\mathcal{D}_c = \{i : \tilde{y}_i = c\}$ be the samples of class c and $\mathcal{C}_c = \mathcal{D}_c \setminus \mathcal{I}$ those among them that were not flagged.

- `naive`: $\mathcal{T} = \{1, \ldots, n\} \setminus \mathcal{I}$, $\mathcal{U} = \mathcal{I}$.
- `topconf`: for each class, the k_c samples of $\mathcal{C}_c$ with the largest s_i (self-confidence) form $\mathcal{T}$, and all the rest form $\mathcal{U}$.
- `random`: for each class, k_c samples drawn at random from $\mathcal{C}_c$ form $\mathcal{T}$, and all the rest form $\mathcal{U}$.

Here $k_c = \min(\max(1, \text{round}(\tau|\mathcal{D}_c|)), |\mathcal{C}_c|)$ and $\tau = 0.3$. In our experiments $|\mathcal{C}_c| \geq \tau|\mathcal{D}_c|$ holds in almost all conditions, so $\mathcal{T}$ of `topconf` and `random` covers about 30% of the training data. In contrast, $\mathcal{T}$ of `naive` covers 53–85% (LR) and 53–83% (HGB) of the training data depending on the condition. `topconf` and `random` preserve the class proportions and are designed so that $\mathcal{U}$ is not made up only of flagged samples. `topconf` lowers the error rate of the trusted set at the cost of stronger selection bias, and `random` does

the opposite. In both, however, all of $\mathcal{I}$ goes into $\mathcal{U}$, so the error rate of $\mathcal{U}$ exceeds the overall noise rate ρ (Section 5.4).

4 Experimental Setup

Datasets We use three datasets: digits (1,797 samples, 64 features, 10 classes); synth, synthetic data from `make_classification` (`n_samples=8000, n_features=20, n_informative=10, n_redundant=4, n_classes=6, n_clusters_per_class=1, class_sep=1.0, random_state=0`); and 12,000 samples drawn by stratified sampling from covtype [15] (54 features of which 44 are binary, 7 classes, strongly imbalanced). For each seed, each dataset is split with stratification into 70% training and 30% test data. Standardization is fitted once on the whole 70% training split before cross-validation and applied to all features, including the binary features of covtype. Noise is added only to the training data; test labels are always the true labels.

Noise For a noise rate $\rho \in \{0.2, 0.4\}$, we flip exactly round(ρn) labels to a different class.

- symmetric: randomly chosen samples are flipped to a uniformly random other class.
- pair: a single random permutation π without fixed points is fixed, and randomly chosen samples are flipped to $\pi(y)$.
- instance: using the predicted probabilities on the training data of a logistic regression f^* (`max_iter=2000`) trained on the true labels of the 70% training split, samples are chosen by weighted sampling without replacement with probability proportional to $1 - f^*(x)_y + 10^{-3}$ and flipped to $\arg\max_{k \neq y} f^*(x)_k$. As with real annotation errors, the more confusing a sample is, the more likely it is to be mislabeled, and as a confusable class.

Because the instance noise is generated with LR, it concentrates near the decision boundary of LR. This noise is hard to detect when the base model is LR, so the conditions are not symmetric between LR and HGB (Section 7).

Models and implementation The base models are logistic regression (LR, `max_iter=2000`) and HistGradientBoosting (HGB, `max_iter=200`, learning rate 0.05). We originally changed the HGB learning rate to 0.05 because training with the default of 0.1 was unstable on covtype. This decision was made by looking at the accuracy with true labels on the test data of the seed-0 split (0.61 at 0.1, 0.79 at 0.05), so the choice referred to the test data. We therefore repeated the comparison post hoc with 5-fold cross-validation inside the training split of each seed (true labels, no test data; Table 11 in the appendix). On covtype the mean accuracy was 76.1% at a learning rate of 0.1 (minimum 72.1%) and 79.1% at 0.05, confirming that the same choice is made without the test data. On digits and synth the difference was at most 0.4 pt. Note that this choice is a check of the base model's stability without noise, not a per-method tuning of the noise-handling methods.

The two probabilistic classifiers of IRBL and the classifier fitted on $\mathcal{U}$ in the `gold` estimator are the base model itself (LR, or HGB with learning rate 0.05). K-PDR uses logistic regression for density-ratio estimation regardless of the base model. K-KMM uses a batch size of 1,000. All other hyperparameters are left at their defaults without tuning. The implementation uses Python 3.11, cleanlab 2.9.0, biquality-learn 0.1.0, and scikit-learn 1.5.2. Because biquality-learn 0.1.0 does not work with scikit-learn 1.6 or later, scikit-learn is pinned to the 1.5 series.

Evaluation For each of the 90 conditions (3 datasets $\times$ 3 noise kinds $\times$ 2 noise rates $\times$ 5 seeds), we compute the test accuracy and its difference Δ (in points) from `baseline` under the same condition. The tables report the mean of Δ over the 90 conditions, the half-width of its 95% confidence interval (1.96$\times$ standard error), and the fraction of conditions with $\Delta > 0$ (win rate). Since the conditions are not independent of each other, the confidence intervals should be read as rough guides.

5 Results

5.1 Overall comparison

Table 1 and Figure 1 show the averages over all conditions. The mean baseline accuracy was 73.14% for LR and 76.93% for HGB. Among the methods that use no human input, `bq/naive/irbl` had the largest Δ for

Table 1: Means over all 90 conditions. Acc. is the test accuracy (%), Δ is the difference from `baseline` (pt) with the half-width of its 95% confidence interval, and Win is the percentage of conditions with $\Delta > 0$. Bold marks the method with the largest Δ excluding the references. `bq/naive/trusted_only` is identical to `cl_prune` and is omitted.

		LR			HGB		
Group	Method	Acc.	Δ (95% CI)	Win	Acc.	Δ (95% CI)	Win
Ref.	`oracle_clean`	81.10	+7.95 ±1.98	99	88.87	+11.95 ±1.81	100
	`oracle_trusted10/irbl`	77.62	+4.48 ±1.91	61	82.56	+5.64 ±1.74	77
	`oracle_trusted30/irbl`	79.78	+6.64 ±1.94	80	85.48	+8.56 ±1.80	94
(1)	`baseline`	73.14	+0.00	–	76.93	+0.00	–
(2)	`cl_prune`	75.43	+2.28 ±0.65	82	80.18	+3.25 ±0.72	83
	`cl_cleanlearning`	74.67	+1.53 ±0.71	66	80.12	+3.19 ±0.70	82
(3) naive	`bq/naive/irbl`	75.87	**+2.73** ±0.79	82	80.49	**+3.56** ±0.65	90
	`bq/naive/kpdr`	75.00	+1.85 ±0.85	57	79.32	+2.40 ±0.53	84
	`bq/naive/kkmm`	74.39	+1.24 ±0.82	43	79.95	+3.03 ±0.64	86
	`bq/naive/glc`	48.78	−24.36 ±4.40	0	40.24	−36.69 ±4.80	0
	`bq/naive/plugin`	42.77	−30.37 ±4.65	6	39.70	−37.23 ±5.63	10
	`bq/naive/irlnl`	46.29	−26.86 ±4.66	2	40.58	−36.34 ±6.25	7
(3) random	`bq/random/trusted_only`	74.92	+1.77 ±0.77	58	78.23	+1.31 ±0.81	56
	`bq/random/irbl`	75.49	+2.35 ±0.85	63	80.14	+3.22 ±0.79	81
	`bq/random/kpdr`	75.00	+1.86 ±0.78	61	78.88	+1.95 ±0.44	84
	`bq/random/kkmm`	74.20	+1.05 ±0.70	50	78.67	+1.74 ±0.65	66
	`bq/random/glc`	68.78	−4.37 ±1.87	21	65.08	−11.84 ±1.72	1
	`bq/random/plugin`	64.25	−8.89 ±2.61	3	65.11	−11.81 ±2.76	0
	`bq/random/irlnl`	63.88	−9.27 ±2.02	7	74.61	−2.32 ±1.93	61
(3) topconf	`bq/topconf/trusted_only`	72.79	−0.35 ±1.43	43	75.99	−0.93 ±1.16	39
	`bq/topconf/irbl`	73.84	+0.70 ±1.39	47	79.34	+2.41 ±0.97	67
	`bq/topconf/kpdr`	72.00	−1.15 ±1.44	36	78.76	+1.83 ±0.68	63
	`bq/topconf/kkmm`	71.78	−1.37 ±1.15	30	77.53	+0.60 ±1.03	48
	`bq/topconf/glc`	55.58	−17.57 ±4.43	12	61.81	−15.12 ±2.72	1
	`bq/topconf/plugin`	56.79	−16.36 ±4.49	9	61.74	−15.18 ±3.45	1
	`bq/topconf/irlnl`	55.28	−17.86 ±4.52	19	73.14	−3.78 ±2.36	58
(4)	`cl_weight/self_confidence`	75.46	+2.32 ±0.58	89	78.92	+1.99 ±0.36	87
	`cl_weight/normalized_margin`	74.98	+1.83 ±0.51	81	78.68	+1.76 ±0.31	87
	`cl_feature`	67.28	−5.86 ±1.23	11	80.38	+3.46 ±0.91	80

both models (LR +2.73, HGB +3.56), followed by `cl_prune` (LR +2.28, HGB +3.25). `cl_cleanlearning` reached LR +1.53 and HGB +3.19, not exceeding `cl_prune`. In contrast, the transition-matrix methods degraded sharply with the `naive` split: LR −24.36 to −30.37, HGB −36.34 to −37.23. The reference `oracle_trusted10/irbl` reached LR +4.48 and HGB +5.64, and `oracle_trusted30/irbl` reached LR +6.64 and HGB +8.56.

5.2 Paired comparisons

Table 2 shows accuracy differences between methods under identical conditions. `bq/naive/irbl` outperformed `cl_prune` by +0.45 pt for LR and +0.31 pt for HGB on average, with win rates of 64% and 71%, respectively. The differences are small, but the confidence intervals exclude 0 for both models. The training time was 2.7 times that of `cl_prune` for LR and 2.9 times for HGB.

Relative to `cl_prune`, `cl_cleanlearning` scored −0.75 pt for LR (win rate 17%) and −0.06 pt for HGB (win rate 37%). Per-class reweighting lowered accuracy for LR and had almost no effect for HGB. As a result, the gap between `bq/naive/irbl` and `cl_cleanlearning` (LR +1.20, HGB +0.37 pt) is larger than that to `cl_prune`.

Among the splitting schemes, `random` consistently outperformed `topconf` (with IRBL, LR +1.65 and HGB +0.80 pt; with `trusted_only`, LR +2.13 and HGB +2.24 pt). `naive` slightly outperformed `random` (with IRBL, LR +0.38 and HGB +0.34 pt). Looking at the gain of IRBL over training on $\mathcal{T}$ alone within each split, the gain with `random` is LR +0.58 and HGB +1.91 pt, larger than the gain with `naive` (the difference from `cl_prune`). In other words, the advantage of `naive` is due less to IRBL's weighting than to the larger size of $\mathcal{T}$ (Section 6).

Using the quality score as weights, `cl_weight/self_confidence` was on par with `cl_prune` for LR (+0.04) and worse for HGB (−1.26). Using it as a feature, `cl_feature` was on par with `cl_prune` for

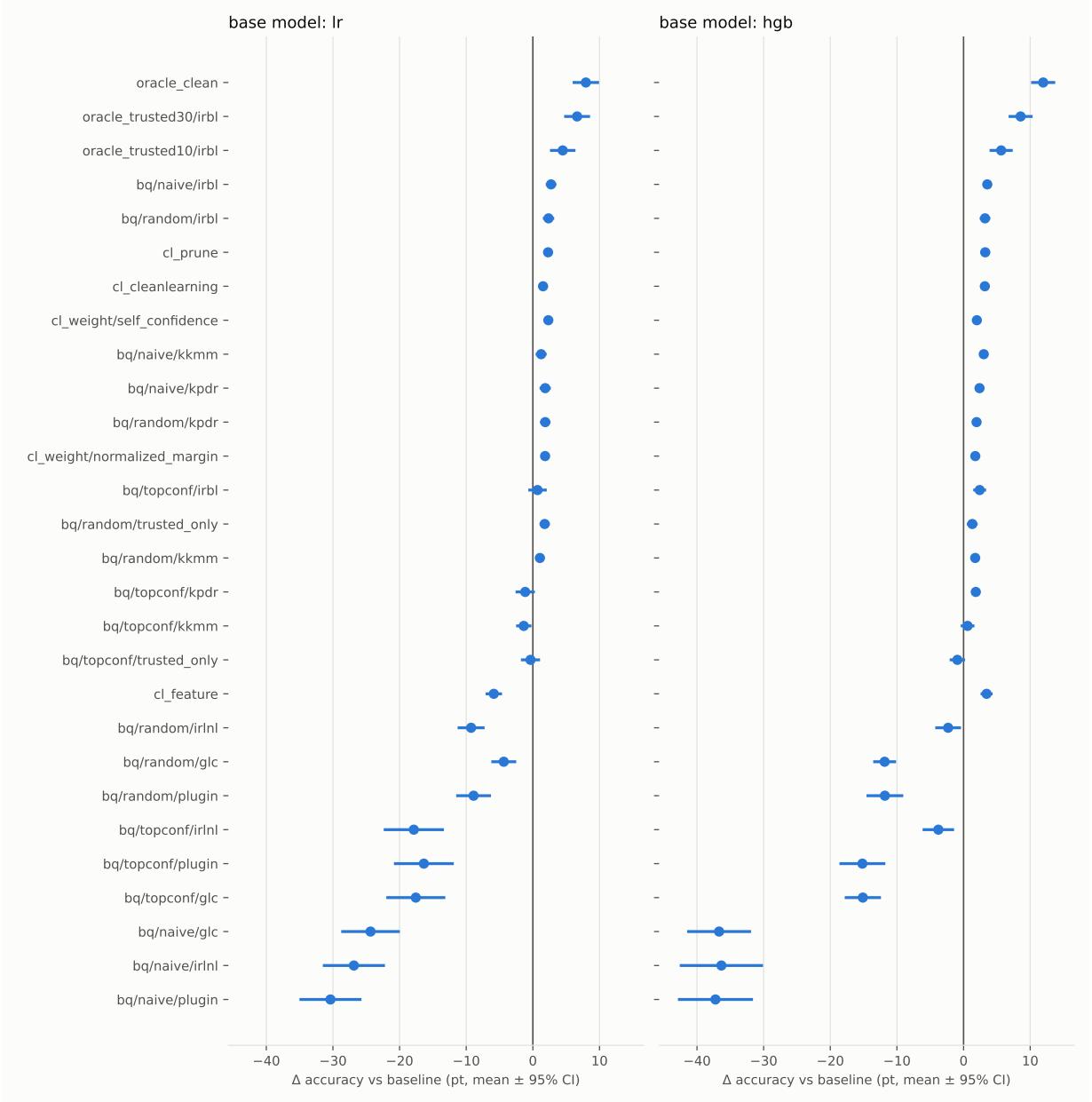


Figure 1: Mean accuracy difference from `baseline` with 95% confidence intervals (left: LR, right: HGB).

Table 2: Accuracy differences (pt) between method A and method B under identical conditions. A win is the percentage of conditions in which A outperformed B. Differences are computed from unrounded accuracies, so they may differ by 0.01 pt from the differences of the rounded values in Table 1.

		LR		HGB	
Method A	Method B	A−B (95% CI)	A win	A−B (95% CI)	A win
`bq/naive/irbl`	`cl_prune`	+0.45 ±0.36	64	+0.31 ±0.24	71
`bq/naive/irbl`	`cl_cleanlearning`	+1.20 ±0.40	76	+0.37 ±0.24	72
`cl_cleanlearning`	`cl_prune`	−0.75 ±0.22	17	−0.06 ±0.17	37
`bq/random/irbl`	`bq/random/trusted_only`	+0.58 ±0.27	80	+1.91 ±0.33	88
`bq/naive/irbl`	`bq/random/irbl`	+0.38 ±0.24	66	+0.34 ±0.28	68
`bq/random/irbl`	`bq/topconf/irbl`	+1.65 ±0.76	72	+0.80 ±0.35	69
`bq/random/trusted_only`	`bq/topconf/trusted_only`	+2.13 ±0.85	70	+2.24 ±0.50	86
`cl_weight/self_confidence`	`cl_prune`	+0.04 ±0.28	57	−1.26 ±0.47	30
`cl_feature`	`cl_prune`	−8.14 ±1.14	2	+0.21 ±0.50	47
`oracle_trusted10/irbl`	`bq/naive/irbl`	+1.75 ±1.32	41	+2.08 ±1.29	44
`oracle_trusted30/irbl`	`bq/naive/irbl`	+3.91 ±1.35	72	+5.00 ±1.38	92
`oracle_trusted30/irbl`	`oracle_trusted10/irbl`	+2.16 ±0.32	99	+2.92 ±0.58	96

HGB (+0.21) but much worse for LR (−8.14).

5.3 Differences by noise kind

Table 3 shows Δ by noise kind and rate. The automatic methods improved most under pair noise: at a noise rate of 40%, `bq/naive/irbl` gained +6.91 pt for LR and +7.48 pt for HGB. Under the same condition, however, `oracle_trusted10/irbl` gained +19.94 (LR) and +21.40 pt (HGB), and `oracle_trusted30/irbl` gained +21.95 (LR) and +24.22 pt (HGB), showing that using human-verified samples as the trusted set is extremely effective. Under instance noise the automatic methods improved little; for LR, no method gained more than 1 pt. For LR at instance 40%, `bq/naive/irbl` (−0.18) fell below `cl_prune` (+0.76) (see Section 5.6 for why). The shortfall of `cl_cleanlearning` relative to `cl_prune` was largest for LR under symmetric noise (+0.68 vs. +2.21 at 40%).

On average over all conditions `oracle_trusted10/irbl` outperformed `bq/naive/irbl`, but its per-condition win rate was below half (LR 41%, HGB 44%), and most of the mean difference comes from pair noise at 40%. In contrast, with the trusted-set size matched at 30%, `oracle_trusted30/irbl` outperformed `bq/naive/irbl` by +3.91 pt for LR (win rate 72%) and +5.00 pt for HGB (win rate 92%). Increasing the trusted set from 10% to 30% improved accuracy in almost every condition (win rate LR 99%, HGB 96%).

5.4 Detection performance of cleanlab and quality of the splits

Table 4 shows the detection performance of cleanlab and the error rates within each split. Recall was high, 84–91%, under symmetric noise and pair noise at 20%, but only 37–74% under instance noise (range of means per dataset and model). The error rate of the `naive` untrusted set (= the flagged samples) was 39–92%, and 2–26% of the labels in the `naive` trusted set were still wrong, depending on the condition. The untrusted sets of `random` and `topconf` also have error rates above ρ because all of $\mathcal{I}$ goes into them (23–28% at $\rho = 20\%$ and 45–54% at $\rho = 40\%$). The trusted set of `topconf` has a lower error rate than that of `random` (e.g., 1.2% vs. 3.1% for HGB with symmetric noise at 20%). Nevertheless, as shown above, `random` achieved higher accuracy.

5.5 Control experiment: genuine trusted data

To separate the degradation caused by pseudo trusted data from the performance of the methods themselves, we restored the true labels of 10% or 30% of the samples chosen uniformly at random, used them as the trusted set, and evaluated the same algorithms (Table 5). The weighting methods IRBL and K-PDR performed best with oracle data and kept positive gains with naive and random. `trusted_only` reached only −0.16 for HGB with 10% oracle data, but LR +6.47 and HGB +6.81 with 30%, so the size of the trusted set strongly affected the result.

The transition-matrix methods fell below `baseline` in most cases even with oracle data (GLC: LR −2.62 and HGB −11.78 at 10%, LR −0.85 and HGB −6.06 at 30%). The exception was IRLNL with HGB, which improved by +5.08 at 30%. The degradation with `naive` (−24 to −37 pt), however, was far

Table 3: Δ by noise kind and rate (pt, mean over 3 datasets $\times$ 5 seeds). Main methods only. The first row is the accuracy of `baseline` (%). All methods are listed in Table 9 in the appendix.

LR	symmetric		pair		instance	
	20%	40%	20%	40%	20%	40%
baseline acc. (%)	77.91	75.63	73.27	58.20	78.35	75.51
`oracle_clean`	+3.19	+5.47	+7.83	+22.90	+2.74	+5.58
`oracle_trusted10/irbl`	+0.25	+2.26	+5.36	+19.94	−1.33	+0.39
`oracle_trusted30/irbl`	+2.33	+4.37	+7.17	+21.95	+1.07	+2.93
`cl_prune`	+1.61	+2.21	+4.54	+3.87	+0.70	+0.76
`cl_cleanlearning`	+0.64	+0.68	+4.17	+2.92	+0.46	+0.30
`bq/naive/irbl`	+1.92	+2.16	+5.37	+6.91	+0.19	−0.18
`bq/random/irbl`	+1.83	+2.15	+5.25	+5.33	+0.04	−0.50
`bq/topconf/irbl`	−0.38	+0.14	+3.01	+3.26	−1.12	−0.73
`bq/naive/kpdr`	+0.86	+0.74	+4.39	+5.77	−0.48	−0.18
`cl_weight/self_confidence`	+2.31	+2.98	+3.78	+3.17	+1.00	+0.67
`cl_feature`	−5.31	−10.40	−6.07	−8.97	−1.66	−2.76
`bq/random/glc`	−7.05	−6.73	−1.49	−8.55	−1.59	−0.78
`bq/naive/glc`	−38.60	−32.27	−32.78	−25.11	−9.75	−7.66
HGB	symmetric		pair		instance	
	20%	40%	20%	40%	20%	40%
baseline acc. (%)	85.61	78.68	82.35	61.36	81.64	71.91
`oracle_clean`	+3.26	+10.19	+6.52	+27.51	+7.23	+16.96
`oracle_trusted10/irbl`	−0.45	+4.44	+2.61	+21.40	+0.15	+5.68
`oracle_trusted30/irbl`	+1.68	+6.69	+4.18	+24.22	+3.69	+10.90
`cl_prune`	+0.51	+5.23	+2.99	+7.13	+0.46	+3.19
`cl_cleanlearning`	+0.46	+5.07	+2.85	+6.70	+0.83	+3.23
`bq/naive/irbl`	+1.29	+5.27	+3.73	+7.48	+0.92	+2.67
`bq/random/irbl`	+0.76	+4.85	+3.19	+7.36	+0.23	+2.93
`bq/topconf/irbl`	−0.55	+4.11	+2.09	+7.06	−0.83	+2.61
`bq/naive/kpdr`	+0.72	+3.14	+2.89	+6.06	−0.18	+1.74
`cl_weight/self_confidence`	+0.88	+3.61	+2.16	+3.09	+0.78	+1.43
`cl_feature`	−0.46	+4.51	+2.63	+9.97	+0.34	+3.74
`bq/random/glc`	−11.19	−18.30	−16.12	−15.62	−6.65	−3.18
`bq/naive/glc`	−53.12	−40.74	−52.48	−26.30	−26.29	−21.20

Table 4: Detection performance of cleanlab and error rates of the pseudo splits (%, mean over 3 datasets $\times$ 5 seeds). Flagged is the fraction of samples flagged as errors. T is trusted and U is untrusted. For `naive`, $\mathcal{U} = \mathcal{I}$, so the error rate of naive U equals the precision, and the size of naive T is 1 − the flagged fraction.

			cleanlab detection			naive		random		topconf	
Model	Noise	Rate	Flagged	Prec.	Recall	T	U	T	U	T	U
LR	symmetric	20%	28.8	62.9	88.2	3.4	62.9	4.2	26.6	1.7	27.6
	symmetric	40%	42.6	80.1	85.0	10.4	80.1	12.2	51.3	5.7	53.9
	pair	20%	31.3	53.9	83.7	4.8	53.9	5.6	26.0	2.6	27.3
	pair	40%	45.1	58.1	65.4	25.2	58.1	27.9	45.1	23.3	47.0
	instance	20%	18.7	55.5	48.7	12.7	55.5	13.0	23.0	4.8	26.5
	instance	40%	27.8	78.9	55.6	24.4	78.9	25.3	46.3	11.1	52.4
HGB	symmetric	20%	26.0	71.8	91.4	2.4	71.8	3.1	27.1	1.2	27.9
	symmetric	40%	44.4	79.9	88.6	8.2	79.9	10.2	52.4	5.6	54.3
	pair	20%	27.6	63.2	86.4	3.8	63.2	5.2	26.3	2.5	27.4
	pair	40%	45.7	56.3	64.3	26.2	56.3	28.2	45.1	23.4	47.1
	instance	20%	20.7	61.4	61.0	9.9	61.4	9.9	24.3	3.4	27.1
	instance	40%	30.7	73.0	56.2	25.2	73.0	26.5	45.8	14.7	50.8

Table 5: Δ (pt) by how the trusted set is built. naive and random are pseudo trusted sets from cleanlab; oracle10 and oracle30 are genuine trusted sets in which 10% or 30% of the samples, chosen uniformly at random, have their true labels restored. † `bq/naive/trusted_only` is identical to `cl_prune`, so its value is shown.

	LR				HGB			
Algorithm	naive	random	oracle10	oracle30	naive	random	oracle10	oracle30
`trusted_only`	+2.28†	+1.77	+3.45	+6.47	+3.25†	+1.31	−0.16	+6.81
`irbl`	+2.73	+2.35	+4.48	+6.64	+3.56	+3.22	+5.64	+8.56
`kpdr`	+1.85	+1.86	+3.11	+6.14	+2.40	+1.95	+4.49	+7.17
`glc`	−24.36	−4.37	−2.62	−0.85	−36.69	−11.84	−11.78	−6.06
`plugin`	−30.37	−8.89	−2.97	−3.58	−37.23	−11.81	−2.46	−2.23
`irlnl`	−26.86	−9.27	−10.82	−8.93	−36.34	−2.32	−1.31	+5.08

Table 6: Δ (pt) with genuine trusted data, broken down by dataset and noise (mean over 2 noise rates × 5 seeds). sym = symmetric, inst = instance.

		covtype			digits			synth		
Algorithm	trusted	sym	pair	inst	sym	pair	inst	sym	pair	inst
LR										
`irbl`	10%	−0.4	+3.3	−2.1	+4.7	+23.2	+2.9	−0.6	+11.4	−2.2
	30%	+1.2	+5.0	−0.2	+8.0	+25.9	+6.5	+0.9	+12.7	−0.3
`glc`	10%	−4.6	−5.1	−5.1	+0.2	+0.4	+0.2	−4.3	−1.7	−3.6
	30%	−10.1	−2.7	−3.5	+2.3	+10.2	+2.4	−3.4	+0.0	−2.8
`plugin`	10%	−19.6	−8.2	−2.7	−1.5	+2.9	−0.3	−0.7	+3.7	−0.2
	30%	−20.2	−8.7	−2.3	−4.4	+3.6	−2.1	−1.0	+3.2	−0.2
`irlnl`	10%	−4.5	−11.5	−7.9	−13.2	−0.8	−3.0	−21.2	−11.6	−23.7
	30%	−10.1	−11.4	−6.5	−11.3	+6.4	−1.3	−19.1	−8.9	−18.1
HGB										
`irbl`	10%	−1.9	+6.1	−0.8	+4.2	+16.2	+3.9	+3.6	+13.7	+5.6
	30%	+0.9	+7.3	+4.1	+5.5	+19.1	+8.4	+6.1	+16.1	+9.3
`glc`	10%	−14.6	−20.5	−12.5	−12.4	−11.5	−9.5	−5.2	−10.6	−9.1
	30%	−9.2	−8.0	−8.2	−6.7	−2.1	−4.2	−3.0	−7.0	−6.1
`plugin`	10%	−9.5	−10.6	−6.0	−0.4	+2.2	−0.6	−0.9	+3.8	−0.1
	30%	−8.6	−4.4	−1.5	−2.9	−0.7	−1.6	−2.3	+2.2	−0.3
`irlnl`	10%	−1.7	−16.5	−13.1	+2.2	+4.8	+1.9	+2.5	+5.6	+2.5
	30%	+0.1	+3.9	+2.0	+4.1	+11.5	+4.8	+3.9	+10.2	+5.2

larger than with oracle data. That is, the transition-matrix methods are not strong in our setting to begin with, and pseudo trusted data makes them much worse.

We initially hypothesized that the transition-matrix methods are weak even with oracle data because estimation is unstable for minority classes with few trusted samples (covtype). The breakdown by dataset (Table 6) shows that the degradation of Plugin is concentrated on covtype (around −20 pt for LR), consistent with this hypothesis. GLC and IRLNL, however, also degrade for LR on the class-balanced synth (GLC down to −4.3 pt, IRLNL −18 to −24 pt under symmetric and instance noise), which minority classes alone cannot explain. Since GLC's estimate of the diagonal barely changed when the trusted set grew from 10% to 30% (Section 5.6), bias in the estimate rather than the number of samples appears to be the main factor.

5.6 Mechanism analysis

For all 90 conditions and both models, we examined the transition matrices estimated by GLC (Table 7) and the weights IRBL assigned to the untrusted samples (Table 8).

Estimating the transition matrix While the mean diagonal of the true transition matrix is 0.69, the value estimated from `naive` was 0.22 for LR and 0.21 for HGB. Because the `gold` estimator uses a classifier trained on $\mathcal{U}$, the estimate reflects the noise in $\mathcal{U}$ rather than in the whole dataset. The fraction of correct labels in the `naive` $\mathcal{U}$ is only 0.35 for LR and 0.32 for HGB, and the estimate fell even below that. Since $\mathcal{U} = \mathcal{I}$ consists solely of samples whose observed labels disagreed with the model's predictions, a classifier trained on $\mathcal{U}$ tends to learn a mapping opposite to the labels of $\mathcal{T}$. This leads to a large overestimate of the noise, and the correction based on it appears to rewrite predictions in the wrong direction. With `random`, unflagged samples mixed into $\mathcal{U}$ raise its clean rate to 0.64, and the estimate improves to LR 0.47 and HGB 0.59.

Even with genuine trusted data, the estimate fell below the clean rate of $\mathcal{U}$ (0.70): 0.50 for LR and 0.62 for HGB. Increasing the trusted set to 30% left the estimate almost unchanged (LR 0.49, HGB 0.62).

Table 7: Mean diagonal of the transition matrix estimated by GLC (`gold`) and the fraction of correct labels in the untrusted set (mean over 3 datasets × 5 seeds). True is the unweighted average over classes of the diagonal of the true transition matrix of the whole dataset, $\frac{1}{K}\sum_k T_{kk}$. Under instance noise, flips concentrate on error-prone classes, so it does not match the sample-level noise rate ρ.

			Mean diagonal estimated by GLC					Clean rate of U			
Model	Noise	Rate	True	naive	random	topconf	oracle10	oracle30	naive	random	oracle10
LR	symmetric	20%	0.80	0.12	0.42	0.47	0.49	0.49	0.37	0.73	0.80
	symmetric	40%	0.60	0.10	0.27	0.24	0.37	0.37	0.20	0.49	0.60
	pair	20%	0.80	0.19	0.47	0.53	0.50	0.49	0.46	0.74	0.80
	pair	40%	0.60	0.25	0.37	0.39	0.38	0.38	0.42	0.55	0.60
	instance	20%	0.78	0.34	0.68	0.80	0.65	0.64	0.45	0.77	0.80
	instance	40%	0.57	0.36	0.64	0.75	0.60	0.60	0.21	0.54	0.60
HGB	symmetric	20%	0.80	0.11	0.61	0.66	0.68	0.68	0.28	0.73	0.80
	symmetric	40%	0.60	0.11	0.39	0.38	0.53	0.53	0.20	0.48	0.60
	pair	20%	0.80	0.14	0.62	0.68	0.67	0.66	0.37	0.74	0.80
	pair	40%	0.60	0.28	0.45	0.44	0.49	0.49	0.44	0.55	0.60
	instance	20%	0.78	0.31	0.76	0.84	0.72	0.71	0.39	0.76	0.80
	instance	40%	0.57	0.31	0.69	0.77	0.63	0.63	0.27	0.54	0.60

Table 8: Mean weights assigned by IRBL to untrusted samples (mean over 3 datasets × 5 seeds). Noisy and Clean denote untrusted samples whose labels are actually wrong and correct, respectively.

			naive		random		topconf	
Model	Noise	Rate	Noisy	Clean	Noisy	Clean	Noisy	Clean
LR	symmetric	20%	0.04	1.48	0.14	1.42	0.33	1.35
	symmetric	40%	0.09	2.32	0.19	2.04	0.29	1.96
	pair	20%	0.06	2.07	0.18	1.38	0.31	1.36
	pair	40%	0.25	1.81	0.49	1.33	0.52	1.26
	instance	20%	0.65	1.29	0.72	1.04	0.85	1.05
	instance	40%	0.80	2.53	0.74	1.17	0.83	1.21
HGB	symmetric	20%	0.01	0.36	0.03	0.91	0.05	0.82
	symmetric	40%	0.02	0.53	0.05	0.98	0.06	0.92
	pair	20%	0.02	0.46	0.07	0.89	0.09	0.81
	pair	40%	0.17	0.54	0.28	0.75	0.28	0.70
	instance	20%	0.12	0.37	0.31	0.87	0.36	0.80
	instance	40%	0.15	0.48	0.35	0.90	0.35	0.86

The `gold` estimator assumes that the classifier trained on $\mathcal{U}$ accurately represents the posterior of the noisy labels. If the classifier's predicted probabilities are not concentrated on the true class, the diagonal is estimated below 1 even without noise, so the classifier's own error appears to be added on top of the noise. That the underestimate is larger for LR and smaller for the more expressive HGB is consistent with this interpretation. For HGB, however, the accuracy drop of GLC is larger than for LR even though its estimate is close to the true value (Table 5), which the bias of the estimate alone cannot explain. Other factors, such as an interaction with training after the correction, are likely involved, but we have not identified them.

IRBL weights IRBL assigned to the `naive` untrusted samples whose labels are actually wrong a mean weight of 0.32 (LR) and 0.08 (HGB), and to those whose labels are actually correct 1.92 (LR) and 0.46 (HGB). That is, rather than discarding all flagged samples, IRBL recovered cleanlab's false positives (samples flagged as errors even though their labels are correct). This appears to be the reason for its slight advantage over `cl_prune`. The absolute weights differ between models, but the ratio of the weights on correct and wrong labels decreased in the order `naive` (LR 6.1, HGB 5.6), `random` (3.4, 4.9), and `topconf` (2.6, 4.1). With `topconf`, the trusted set is biased toward easy samples, which apparently makes it harder for IRBL to tell wrong labels from "hard but correct" ones, consistent with the results for RQ2.

For LR at instance 40%, the mean weight on the wrong labels in the `naive` untrusted set was 0.80, barely lower than for correct labels and in contrast to symmetric noise (0.04–0.09). Instance noise consists of errors that look natural to LR, and 24% of the labels in the `naive` trusted set remain wrong, so IRBL cannot tell the errors apart. This appears to be why `bq/naive/irbl` fell below `cl_prune` in this condition.

6 Discussion

RQ1: Is pseudo-biquality learning better than pruning? Only in combination with weighting methods, IRBL in particular, did it slightly outperform pruning. The improvement is small, 0.3–0.5 pt, and the training time is about three times longer. cleanlab's standard `CleanLearning` removes the same samples and only adds per-class reweighting, yet it did not outperform `cl_prune`. Since the shortfall was largest for LR under symmetric noise, where the noise rate does not depend on the class, variation in the estimated per-class noise rates may have introduced spurious bias between classes. The advantage of `bq/naive/irbl` therefore cannot be explained by reweighting per se; it appears to lie in recovering falsely flagged samples individually.

The size of the trusted set, however, also contributes to the advantage of `naive`. $\mathcal{T}$ of `naive` covers the majority of the training data, whereas that of `random` covers about 30%. Since the gain of IRBL over training on $\mathcal{T}$ alone was larger with `random`, IRBL's weighting itself works even with a small $\mathcal{T}$, and the benefit of `naive` appears to lie mainly in the size of $\mathcal{T}$. Combinations with transition-matrix methods should be avoided.

RQ2: How to choose the trusted set `topconf`, which uses the samples with the top quality scores as trusted, lowered accuracy even though it lowered the error rate of the trusted set. When the trusted set is biased toward "samples that are easy for the model", weighting methods end up weighting by "easy or hard", which apparently lowers the weights of hard but correct samples. The measured IRBL weights (Table 8) support this. This is also consistent with the finding of Dataset Cartography [8] that confidence-based selection is biased toward easy-to-learn samples and that ambiguous samples contribute to generalization. When building a pseudo trusted set, following the same distribution as the untrusted set should take priority over the cleanliness of the labels.

This tendency, however, depended on the dataset (Table 10 in the appendix). For LR on covtype, which is strongly imbalanced, every split that narrows the trusted set to 30% of the training data fell below `baseline` (`bq/random/irbl` -0.53 and `bq/random/trusted_only` -1.14 pt), and positive gains were limited to `bq/naive/irbl` ($+0.53$) and `cl_prune` ($+0.14$). For LR on digits, by contrast, `bq/topconf/irbl` ($+7.01$) outperformed `bq/random/irbl` ($+5.41$). The disadvantage of narrowing the trusted set and the effect of selection bias appear to interact with the structure and class balance of the data.

RQ3: Using quality scores as weights or features Using the scores as weights was on par with or worse than pruning and showed no benefit. Using them as a feature worked as well as pruning for HGB but degraded LR substantially. We see three possible factors. First, when s is concatenated additively to a multiclass logistic regression, the logit of class c becomes $w_c^\top x + w_{c,s} s + b_c$, so s can only shift the logits independently of x. It therefore cannot represent "the decision boundary when the label is reliable". This is a limitation specific to additive concatenation without interaction terms (such as sx); a linear model can represent it if the interactions are added explicitly. Second, s is spread widely below 1 during training, whereas $s = 1$ is given uniformly at inference. If classes with a high average s during training receive a positive $w_{c,s}$, a constant corresponding to $w_{c,s}(1 - \bar{s})$ is added to each class's intercept at inference, which may bias predictions toward those classes. That LR degraded strongly even under symmetric noise (-5.31 pt at 20% and -10.40 pt at 40%) is consistent with this intercept shift. Third, for HGB, splits on s ($s > \theta$) route inputs at inference, where $s = 1$, into subtrees built from high-confidence training samples. This appears to act as implicit pruning inside the trees rather than learning a decision boundary conditioned on confidence. We have not verified these explanations directly.

Practical implications If no human input is used at all, pruning with cleanlab is the first choice in terms of the balance between ease and effect. In our experiments, the per-class reweighting of `CleanLearning` brought no additional gain. If computation time allows, combining the `naive` split with IRBL can be expected to add a small gain. On the other hand, if 30% of the training data can be verified by hand, IRBL outperforms the best automatic method by about 4–5 pt on average, with a high win rate (LR 72%, HGB 92%). With 10% verification, it wins on average but in fewer than half of the conditions, although under heavy systematic noise (pair 40%) it outperformed the automatic methods by 13–14 pt. When the nature of the noise is unknown, it is worth verifying a small random sample to learn the noise rate and its tendencies.

7 Limitations

- All noise is synthetic. In particular, the instance noise was generated with LR, so it is biased toward directions that are hard for LR to detect, and the conditions are not symmetric between LR and HGB.
- We use only three small-to-medium tabular datasets and do not cover images, text, or deep learning models. The effect of narrowing the trusted set differs between datasets (covtype vs. digits), which limits generalization from three datasets.
- Hyperparameters (τ, the detection settings of cleanlab, and the settings of each algorithm) were not tuned, and only a single value, $\tau = 0.3$, was evaluated.
- The HGB learning rate was originally chosen by referring to the test data. We confirmed post hoc that cross-validation within the training data leads to the same choice, but the selection process itself depended on the test data.
- We confirmed that the underestimate of the `gold` estimator and minority classes contribute to the weakness of the transition-matrix methods with genuine trusted data, but we have not identified what causes the degradation for HGB.
- The training time of `CleanLearning` includes re-running the label-issue detection internally, so it is not comparable with the other methods and is not reported.
- The confidence intervals do not account for correlations between conditions.

8 Conclusion

We evaluated pseudo-biquality learning, which builds trusted/untrusted sets automatically from cleanlab's output and feeds them to biquality-learn, on 90 conditions $\times$ 2 models. The combination with IRBL slightly outperformed pruning and `CleanLearning`, but part of this benefit came from the size of the trusted set. Combinations with transition-matrix methods broke down badly, and we confirmed by measurement that the cause is the underestimated transition matrix resulting from mislabeled samples concentrating in the untrusted set. For choosing the trusted set, low distributional bias mattered more than clean labels. On the other hand, the gap to a human-verified trusted set of the same size was large, so the cost of removing humans entirely is not small. Future work includes validation on real-world noise, a systematic exploration of the trusted-set ratio and selection, and semi-automatic schemes that combine a small amount of human verification with cleanlab.

References

[1] B. Frénay and M. Verleysen. "Classification in the Presence of Label Noise: A Survey". In: *IEEE Transactions on Neural Networks and Learning Systems* 25.5 (2014), pp. 845–869. DOI: 10.1109/TNNLS.2013.2292894.

[2] H. Song et al. "Learning From Noisy Labels With Deep Neural Networks: A Survey". In: *IEEE Transactions on Neural Networks and Learning Systems* 34.11 (2023), pp. 8135–8153. DOI: 10.1109/TNNLS.2022.3152527.

[3] C. G. Northcutt, L. Jiang, and I. L. Chuang. "Confident Learning: Estimating Uncertainty in Dataset Labels". In: *Journal of Artificial Intelligence Research* 70 (2021), pp. 1373–1411. DOI: 10.1613/jair.1.12125.

[4] P. Nodet, V. Lemaire, A. Bondu, and A. Cornuéjols. "Biquality Learning: A Framework to Design Algorithms Dealing with Closed-Set Distribution Shifts". In: *Machine Learning* 112.12 (2023), pp. 4663–4692. DOI: 10.1007/s10994-023-06372-3.

[5] P. Nodet, V. Lemaire, A. Bondu, and A. Cornuéjols. *biquality-learn: a Python library for Biquality Learning*. 2023. arXiv: 2308.09643 [cs.LG].

[6] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[7] T. George, P. Nodet, A. Bondu, and V. Lemaire. "Mislabeled Examples Detection Viewed as Probing Machine Learning Models: Concepts, Survey and Extensive Benchmark". In: *Transactions on Machine Learning Research* (2024). arXiv: 2410.15772.

[8] S. Swayamdipta et al. “Dataset Cartography: Mapping and Diagnosing Datasets with Training Dynamics”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, pp. 9275–9293. DOI: 10.18653/v1/2020.emnlp-main.746.

[9] P. Nodet et al. “Importance Reweighting for Biquality Learning”. In: *2021 International Joint Conference on Neural Networks (IJCNN)*. 2021, pp. 1–8. DOI: 10.1109/IJCNN52387.2021.9533349.

[10] S. Bickel, M. Brückner, and T. Scheffer. “Discriminative Learning for Differing Training and Test Distributions”. In: *Proceedings of the 24th International Conference on Machine Learning*. 2007, pp. 81–88. DOI: 10.1145/1273496.1273507.

[11] J. Huang et al. “Correcting Sample Selection Bias by Unlabeled Data”. In: *Advances in Neural Information Processing Systems 19 (NIPS 2006)*. 2006, pp. 601–608.

[12] D. Hendrycks, M. Mazeika, D. Wilson, and K. Gimpel. “Using Trusted Data to Train Deep Networks on Labels Corrupted by Severe Noise”. In: *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*. 2018, pp. 10456–10465.

[13] M. Zhang, J. Lee, and S. Agarwal. “Learning from Noisy Labels with No Change to the Training Process”. In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. 2021, pp. 12468–12478.

[14] T. Liu and D. Tao. “Classification with Noisy Labels by Importance Reweighting”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 38.3 (2016), pp. 447–461. DOI: 10.1109/TPAMI.2015.2456899.

[15] J. A. Blackard and D. J. Dean. “Comparative Accuracies of Artificial Neural Networks and Discriminant Analysis in Predicting Forest Cover Types from Cartographic Variables”. In: *Computers and Electronics in Agriculture* 24.3 (1999), pp. 131–151. DOI: 10.1016/S0168-1699(99)00046-0.

A Detailed Results

Table 9: Δ by noise kind and rate (pt, all methods). The first row is the accuracy of `baseline` (%).

LR	symmetric		pair		instance	
	20%	40%	20%	40%	20%	40%
baseline acc. (%)	77.91	75.63	73.27	58.20	78.35	75.51
`oracle_clean`	+3.19	+5.47	+7.83	+22.90	+2.74	+5.58
`oracle_trusted10/irbl`	+0.25	+2.26	+5.36	+19.94	−1.33	+0.39
`oracle_trusted30/irbl`	+2.33	+4.37	+7.17	+21.95	+1.07	+2.93
`cl_prune`	+1.61	+2.21	+4.54	+3.87	+0.70	+0.76
`cl_cleanlearning`	+0.64	+0.68	+4.17	+2.92	+0.46	+0.30
`bq/naive/irbl`	+1.92	+2.16	+5.37	+6.91	+0.19	−0.18
`bq/naive/kpdr`	+0.86	+0.74	+4.39	+5.77	−0.48	−0.18
`bq/naive/kkmm`	−0.23	−1.04	+3.80	+5.80	−0.39	−0.49
`bq/naive/glc`	−38.60	−32.27	−32.78	−25.11	−9.75	−7.66
`bq/naive/plugin`	−49.56	−50.07	−18.73	−9.32	−30.34	−24.21
`bq/naive/irlnl`	−46.12	−37.07	−34.85	−23.44	−10.67	−8.99
`bq/random/trusted_only`	+1.16	+1.73	+4.16	+3.11	+0.07	+0.41
`bq/random/irbl`	+1.83	+2.15	+5.25	+5.33	+0.04	−0.50
`bq/random/kpdr`	+1.11	+1.46	+4.58	+4.37	−0.31	−0.04
`bq/random/kkmm`	+0.26	+0.09	+3.97	+2.93	−0.33	−0.59
`bq/random/glc`	−7.05	−6.73	−1.49	−8.55	−1.59	−0.78
`bq/random/plugin`	−14.21	−19.88	−6.63	−9.83	−1.22	−1.61
`bq/random/irlnl`	−15.52	−9.67	−6.85	−9.39	−8.91	−5.27
`bq/topconf/trusted_only`	−2.17	−1.46	+1.34	+0.81	−1.12	+0.48
`bq/topconf/irbl`	−0.38	+0.14	+3.01	+3.26	−1.12	−0.73
`bq/topconf/kpdr`	−2.61	−2.57	−0.15	+1.07	−1.91	−0.71
`bq/topconf/kkmm`	−2.77	−2.87	−0.13	+1.20	−2.38	−1.26
`bq/topconf/glc`	−25.10	−31.27	−18.10	−23.74	−1.03	−6.14
`bq/topconf/plugin`	−20.05	−33.94	−15.53	−21.52	−2.15	−4.93
`bq/topconf/irlnl`	−28.06	−31.56	−14.00	−23.08	−4.49	−6.01
`cl_weight/self_confidence`	+2.31	+2.98	+3.78	+3.17	+1.00	+0.67
`cl_weight/normalized_margin`	+1.55	+1.58	+3.63	+2.80	+0.85	+0.59
`cl_feature`	−5.31	−10.40	−6.07	−8.97	−1.66	−2.76

HGB	symmetric		pair		instance	
	20%	40%	20%	40%	20%	40%
baseline acc. (%)	85.61	78.68	82.35	61.36	81.64	71.91
`oracle_clean`	+3.26	+10.19	+6.52	+27.51	+7.23	+16.96
`oracle_trusted10/irbl`	−0.45	+4.44	+2.61	+21.40	+0.15	+5.68
`oracle_trusted30/irbl`	+1.68	+6.69	+4.18	+24.22	+3.69	+10.90
`cl_prune`	+0.51	+5.23	+2.99	+7.13	+0.46	+3.19
`cl_cleanlearning`	+0.46	+5.07	+2.85	+6.70	+0.83	+3.23
`bq/naive/irbl`	+1.29	+5.27	+3.73	+7.48	+0.92	+2.67
`bq/naive/kpdr`	+0.72	+3.14	+2.89	+6.06	−0.18	+1.74
`bq/naive/kkmm`	+0.64	+4.39	+2.92	+7.13	+0.42	+2.69
`bq/naive/glc`	−53.12	−40.74	−52.48	−26.30	−26.29	−21.20
`bq/naive/plugin`	−62.42	−57.03	−21.08	−5.88	−43.67	−33.31
`bq/naive/irlnl`	−64.59	−64.25	−61.43	−7.68	−9.91	−10.18
`bq/random/trusted_only`	−1.96	+3.05	−0.12	+5.10	−0.81	+2.58
`bq/random/irbl`	+0.76	+4.85	+3.19	+7.36	+0.23	+2.93
`bq/random/kpdr`	+0.44	+2.85	+2.06	+4.92	−0.03	+1.47
`bq/random/kkmm`	−0.38	+2.81	+1.79	+4.80	−0.29	+1.72
`bq/random/glc`	−11.19	−18.30	−16.12	−15.62	−6.65	−3.18
`bq/random/plugin`	−8.28	−30.88	−10.22	−15.05	−2.69	−3.74
`bq/random/irlnl`	−0.47	−8.75	−2.09	−4.06	+0.47	+1.00
`bq/topconf/trusted_only`	−6.02	+0.50	−3.09	+4.79	−4.15	+2.40
`bq/topconf/irbl`	−0.55	+4.11	+2.09	+7.06	−0.83	+2.61
`bq/topconf/kpdr`	−0.26	+2.74	+1.49	+5.90	−0.99	+2.12
`bq/topconf/kkmm`	−3.31	+1.95	−0.51	+5.64	−2.65	+2.50
`bq/topconf/glc`	−13.33	−24.31	−19.80	−21.87	−5.44	−5.95
`bq/topconf/plugin`	−9.63	−33.35	−17.82	−24.21	−1.29	−4.80
`bq/topconf/irlnl`	−0.63	−9.35	−4.21	−10.14	+0.60	+1.03
`cl_weight/self_confidence`	+0.88	+3.61	+2.16	+3.09	+0.78	+1.43
`cl_weight/normalized_margin`	+0.48	+2.49	+2.09	+3.21	+0.65	+1.63
`cl_feature`	−0.46	+4.51	+2.63	+9.97	+0.34	+3.74

Table 10: Δ by dataset (pt, all methods, mean over 3 noise kinds $\times$ 2 rates $\times$ 5 seeds). The first row is the accuracy of `baseline` (%).

LR	covtype	digits	synth
baseline acc. (%)	69.71	80.95	68.77
`oracle_clean`	+2.84	+15.86	+5.15
`oracle_trusted10/irbl`	+0.29	+10.30	+2.85
`oracle_trusted30/irbl`	+2.02	+13.47	+4.42
`cl_prune`	+0.14	+5.48	+1.22
`cl_cleanlearning`	−1.42	+4.78	+1.22
`bq/naive/irbl`	+0.53	+5.22	+2.43
`bq/naive/kpdr`	−0.43	+5.54	+0.45
`bq/naive/kkmm`	−1.22	+3.57	+1.38
`bq/naive/glc`	−10.10	−36.83	−26.15
`bq/naive/plugin`	−29.51	−40.50	−21.11
`bq/naive/irlnl`	−15.51	−38.06	−27.00
`bq/random/trusted_only`	−1.14	+5.37	+1.09
`bq/random/irbl`	−0.53	+5.41	+2.17
`bq/random/kpdr`	−0.69	+4.48	+1.79
`bq/random/kkmm`	−1.14	+2.85	+1.46
`bq/random/glc`	−9.26	+0.90	−4.73
`bq/random/plugin`	−19.15	−4.54	−2.99
`bq/random/irlnl`	−12.09	−4.92	−10.79
`bq/topconf/trusted_only`	−7.49	+6.73	−0.31
`bq/topconf/irbl`	−5.89	+7.01	+0.98
`bq/topconf/kpdr`	−7.53	+5.72	−1.63
`bq/topconf/kkmm`	−6.64	+3.33	−0.79
`bq/topconf/glc`	−37.62	−2.01	−13.07
`bq/topconf/plugin`	−37.62	−2.03	−9.42
`bq/topconf/irlnl`	−38.02	−1.59	−13.99
`cl_weight/self_confidence`	+0.41	+5.35	+1.20
`cl_weight/normalized_margin`	+0.18	+4.32	+1.00
`cl_feature`	−11.10	−0.68	−5.81
HGB	covtype	digits	synth
baseline acc. (%)	70.91	83.47	76.40
`oracle_clean`	+8.87	+13.64	+13.33
`oracle_trusted10/irbl`	+1.15	+8.11	+7.65
`oracle_trusted30/irbl`	+4.14	+11.03	+10.51
`cl_prune`	+0.64	+4.98	+4.13
`cl_cleanlearning`	+0.51	+5.04	+4.02
`bq/naive/irbl`	+1.36	+4.91	+4.41
`bq/naive/kpdr`	+1.57	+3.04	+2.57
`bq/naive/kkmm`	+1.04	+4.51	+3.54
`bq/naive/glc`	−21.12	−53.56	−35.39
`bq/naive/plugin`	−31.28	−46.13	−34.28
`bq/naive/irlnl`	−31.70	−41.78	−35.54
`bq/random/trusted_only`	−1.44	+3.09	+2.27
`bq/random/irbl`	+0.23	+5.14	+4.29
`bq/random/kpdr`	+0.93	+2.43	+2.49
`bq/random/kkmm`	−0.77	+3.16	+2.84
`bq/random/glc`	−16.01	−11.02	−8.50
`bq/random/plugin`	−20.98	−8.23	−6.22
`bq/random/irlnl`	−8.68	+0.83	+0.90
`bq/topconf/trusted_only`	−3.79	+0.81	+0.18
`bq/topconf/irbl`	−1.46	+5.01	+3.70
`bq/topconf/kpdr`	−0.26	+3.10	+2.65
`bq/topconf/kkmm`	−2.86	+3.04	+1.62
`bq/topconf/glc`	−20.02	−17.22	−8.12
`bq/topconf/plugin`	−26.71	−9.54	−9.30
`bq/topconf/irlnl`	−11.78	−0.25	+0.67
`cl_weight/self_confidence`	+0.89	+2.99	+2.10
`cl_weight/normalized_margin`	+0.77	+2.61	+1.89
`cl_feature`	+1.00	+5.98	+3.39

Table 11: Check of the HGB learning rate: accuracy (%) of 5-fold cross-validation within the training split of each seed (true labels, no test data). Mean (min–max) over 5 seeds.

Dataset	Learning rate 0.1	Learning rate 0.05
covtype	76.1 (72.1–79.7)	79.1 (78.8–79.7)
digits	96.8 (96.3–97.2)	96.8 (96.4–97.1)
synth	89.4 (89.0–89.9)	89.0 (88.6–89.6)

B Reproduction

The code is in `label-noise-experiment/`. The following commands run all experiments in this paper and regenerate the tables, the figure, and the numbers in the text. Figure 1, all tables, and the macros for the numbers in the text (`latex/numbers.tex`) are generated by `export_latex`. The loop (for both models) and `hgb_lr_check` can also be run together with `sh rerun_all.sh`.

```
for m in lr hgb; do
  uv run python -m label_noise_experiment.run --model $m
  uv run python -m label_noise_experiment.oracle_check --model $m --oracle-frac 0.1
  uv run python -m label_noise_experiment.oracle_check --model $m --oracle-frac 0.3
  uv run python -m label_noise_experiment.mechanism --model $m
done
uv run python -m label_noise_experiment.hgb_lr_check
uv run python -m label_noise_experiment.export_latex
wsl sh latex/build.sh    # build the PDFs (Docker image texlive/texlive)
```